	ESCOLA TÉCNICA SATC **Curso:** Técnico em Mecatrônica **Fase**: 3ª **Componente Curricular:** Sistemas Embarcados **Objetivo**: Laços de Repetição. **Professor:** Marcos Antonio Jeremias Coelho **Aluno(a):** _____ **Data:**	**Peso** **N.A.** **Nota** _____

LAÇOS DE REPETIÇÃO E VETORES EM LINGUAGEM C PARA ARDUINO

1. OBJETIVOS

Nesta aula o discente irá compreender dois pilares fundamentais da programação aplicada a sistemas embarcados: as estruturas de repetição e os vetores. Esses recursos são indispensáveis para quem deseja programar Arduino, ESP32, CLPs modernos e sistemas de automação em geral.

Ao final do material, espera-se que o discente consiga identificar situações em que tarefas precisam ser repetidas automaticamente, escolher o tipo correto de laço de repetição e utilizar *arrays* para organizar múltiplos dados de forma profissional e escalável.

2. INTRODUÇÃO

Na prática da mecatrônica, raramente um sistema executa apenas uma ação isolada. Normalmente o equipamento trabalha de forma contínua, repetitiva e automática. Um sensor mede temperatura diversas vezes por minuto, um motor gira até atingir determinada posição, uma esteira verifica presença de peças repetidamente e um semáforo executa o mesmo ciclo ao longo do dia inteiro.

Se o programador tentar escrever cada repetição manualmente, o código rapidamente se torna grande, confuso e difícil de manter. Além disso, qualquer alteração futura exigirá retrabalho. É justamente para resolver esse problema que

existem os laços de repetição e os vetores. Essas ferramentas permitem construir programas mais limpos, menores e muito mais eficientes.

3. ESTRUTURAS DE REPETIÇÃO

Estruturas de repetição são comandos usados quando desejamos executar um bloco de código várias vezes. Em vez de escrever a mesma instrução repetidamente, o programador define uma regra e o compilador executa esse bloco quantas vezes forem necessárias.

Na linguagem C utilizada no Arduino, os principais laços são:

- `for`
- `while`
- `do...while`

Cada um deles possui finalidade específica. Saber escolher corretamente demonstra maturidade técnica.

3.1. LAÇO `FOR`

O laço for é indicado quando o número de repetições já é conhecido antes do início da execução. Isso acontece frequentemente em sistemas embarcados: piscar um LED dez vezes, testar cinco sensores, mover um motor vinte passos ou repetir uma leitura determinada quantidade de vezes.

Sua estrutura clássica é:

```
for(inicialização; condição; incremento) {
   comandos;
}
```

A primeira parte inicializa uma variável de controle, normalmente chamada de contador. A segunda parte informa até quando o laço deve continuar. A terceira parte atualiza o contador a cada repetição.

Exemplo:

```
for(int i=0; i<5; i++){
   Serial.println(i);
}
```

Nesse caso, a variável *i* começa em zero. Enquanto *i* for menor que cinco, o código será executado. Ao final de cada ciclo, *i* recebe incremento de uma unidade. O resultado exibido será de 0 até 4.

Esse tipo de estrutura é extremamente útil porque concentra toda a lógica de repetição em uma única linha. O código fica compacto e claro.

3.1.1. Exemplo aplicado ao arduino

```
int led = 13;

void setup() {
  pinMode(led, OUTPUT);
  for(int i=0; i<10; i++){
    digitalWrite(led, HIGH);
    delay(500);
    digitalWrite(led, LOW);
    delay(500);
  }
}

void loop() {
}
```

Nesse programa, o LED interno do Arduino irá piscar dez vezes. Observe que seria improdutivo escrever dez blocos iguais manualmente. O *for* resolve isso de forma elegante.

3.1.2. Quando Utilizar

Use for sempre que a quantidade de repetições for previsível. Em engenharia, isso é comum em rotinas de teste, calibração, varredura de portas e acionamentos em sequência.

3.2. LAÇO `WHILE`

O laço *while* é usado quando não sabemos exatamente quantas repetições serão necessárias. Em vez de contar ciclos, ele continua executando enquanto determinada condição permanecer verdadeira.

Sua estrutura é:

```
while(condição){
   comandos;
}
```

Isso significa que antes de cada repetição o sistema verifica a condição. Se ela for verdadeira, executa novamente. Se for falsa, sai do laço.

Exemplo:

```
int contador = 0;

while(contador < 5){
   Serial.println(contador);
   contador++;
}
```

Nesse caso, o laço continuará funcionando até que a variável alcance o valor cinco.

3.2.1. Aplicação real em sistemas embarcados

Muitas vezes o microcontrolador precisa aguardar um evento externo, como o pressionamento de um botão, chegada de sinal serial, conexão Wi-Fi ou leitura de sensor. Nesses casos, o *while* é bastante apropriado.

```
int botao = 2;

void setup() {
  pinMode(botao, INPUT);
  Serial.begin(9600);

  while(digitalRead(botao) == LOW){
    Serial.println("Aguardando botão...");
    delay(500);
  }

  Serial.println("Botão pressionado");
}

void loop() {
}
```

O sistema permanece aguardando até que o botão seja acionado. Esse comportamento é comum em máquinas industriais que esperam comando do operador.

3.2.2. Cuidado técnico

Se a condição nunca mudar, o laço entra em repetição infinita. Em sistemas embarcados isso pode travar o funcionamento. Portanto, sempre verifique se existe alguma forma real de a condição ser alterada.

3.3. LAÇO `DO...WHILE`

O *do...while* possui funcionamento semelhante ao *while*, porém com uma diferença importante: o bloco de comandos é executado primeiro e a condição só é testada depois. Isso garante pelo menos uma execução.

Estrutura:

```
do{
   comandos;
}while(condição);
```

Exemplo:

```
int x = 0;

do{
   Serial.println(x);
   x++;
}while(x < 5);
```

Nesse caso, o programa imprime valores de 0 a 4.

3.3.1. Diferença prática entre `while` e `do...while`

Se a condição já começar falsa:

```
while(false){
}
```

Nada será executado.

Já no *do...while*:

```
do{
   Serial.println("Executou");
}while(false);
```

O comando roda uma vez.

3.3.2. Aplicações

Esse laço é útil em inicializações obrigatórias, menus de interface, tentativas de conexão e rotinas que precisam acontecer ao menos uma vez antes da verificação.

```
int tentativa = 0;

void setup() {
  Serial.begin(9600);

  do{
    Serial.println("Tentando conectar...");
    tentativa++;
    delay(1000);
  }while(tentativa < 3);
}

void loop() {
}
```

Nesse exemplo, o sistema tenta conectar três vezes.

4. VETORES (*ARRAYS*)

Em programação, vetor é uma estrutura capaz de armazenar vários valores do mesmo tipo dentro de uma única variável organizada por posições.

Imagine que um projeto tenha três LEDs conectados nos pinos 8, 9 e 10.

Sem vetor:

```
const int led1 = 8;
const int led2 = 9;
const int led3 = 10;
```

Com vetor:

```
const int leds[3] = {8,9,10};
```

A segunda forma é muito superior porque organiza melhor o projeto e facilita expansão futura.

4.1. ÍNDICES

As posições do vetor começam em zero:

- `leds[0]` vale 8
- `leds[1]` vale 9
- `leds[2]` vale 10

Esse detalhe é padrão da linguagem C e precisa ser respeitado.

4.2. USO DE *ARRAYS* PARA SIMPLIFICAR CÓDIGO

Quando combinamos *arrays* com laços de repetição, a programação embarcada evolui de nível.

4.2.1. Exemplo de configuração de vários LEDs:

```
int leds[3] = {8,9,10};

void setup() {

  for(int i=0; i<3; i++){
    pinMode(leds[i], OUTPUT);
  }
}
```

Em vez de escrever três linhas separadas para *pinMode*, o laço percorre o vetor automaticamente.

4.2.2. Sequência de acionamento

```
int leds[3] = {8,9,10};

void setup() {
  for(int i=0; i<3; i++){
    pinMode(leds[i], OUTPUT);
  }
}

void loop() {
  for(int i=0; i<3; i++){
    digitalWrite(leds[i], HIGH);
    delay(300);
    digitalWrite(leds[i], LOW);
  }
}
```

Esse tipo de lógica aparece em sinalização, painéis luminosos e sistemas sequenciais.

5. APLICAÇÕES REAIS

Os vetores não servem apenas para LEDs. Em projetos profissionais, são usados para sensores, relés, motores e armazenamento de medições.

5.1. SENSORES ANALÓGICOS

```
int sensores[4] = {A0,A1,A2,A3};
```

Leitura automática:

```
for(int i=0; i<4; i++){
   Serial.println(analogRead(sensores[i]));
}
```

5.2. BANCO DE RELÉS

```
int reles[4] = {4,5,6,7};
```

Ligar todos:

```
for(int i=0; i<4; i++){
   digitalWrite(reles[i], HIGH);
}
```

Essa lógica é comum em automação predial, painéis elétricos e controle industrial.

6. ERROS FREQUENTES

Um erro clássico é esquecer de atualizar a variável de controle no *while*, gerando loop infinito.

Outro erro comum é acessar posições inexistentes do vetor.

```
int v[3];
v[3] = 10;
```

Está errado, pois as posições válidas são:

```
v[0]
v[1]
v[2]
```

Falhas desse tipo podem gerar comportamento imprevisível no microcontrolador.

7. EXERCÍCIOS

1. Criar um programa que faça um LED piscar vinte vezes usando for.
2. Criar um sistema que aguarde botão pressionado usando *while*.
3. Montar vetor com cinco LEDs e acender um por vez.
4. Ler quatro sensores analógicos e exibir valores no Serial Monitor.
5. Desenvolver semáforo automático usando *arrays* e repetição.